The Rainbow Series in Full: A Thematic Tutorial on Trusted Systems Guidance and Its Modern Successors

Made by AI

***Abstract*—The Rainbow Series, a set of more than thirty computer security guidelines published by the U.S. National Computer Security Center between 1983 and the mid-1990s, established much of the vocabulary of trusted computing. This tutorial explains the whole series. Part I covers the eight volumes whose ideas remain most influential: the Trusted Computer System Evaluation Criteria (the Orange Book), its glossary, and the guides on security modeling, discretionary access control, audit, object reuse, data remanence, and covert channel analysis. Part II covers the remaining volumes, grouped into eight themes: identification and authentication, networks, layered systems such as databases, recovery, the software supply chain, the assurance process, operations, and governance. For each topic we first explain the core idea in plain terms, with concrete examples, and then summarize the essential modern developments, including the Common Criteria, formally verified kernels, current NIST guidance, zero trust networking, passkeys, supply-chain frameworks, and microarchitectural side-channel attacks. We then identify the areas the series did not anticipate. We conclude that the architectural principles of the series have aged well, while its evaluation process and several of its specific rules have not. A reading path, a glossary, and an index of every volume are provided for readers who wish to study the original documents.**

***Index Terms*—Access control, accreditation, assurance, audit, authentication, covert channels, data remanence, Orange Book, security models, software supply chain, trusted computing base.**

I. Introduction

In August 1983 the U.S. Department of Defense Computer Security Center published the *Trusted Computer System Evaluation Criteria* (TCSEC), reissued in December 1985 as DoD 5200.28-STD [1]. Because of the color of its cover, the document became known as the Orange Book. Over the following decade the center, later the National Computer Security Center (NCSC), issued more than thirty companion guidelines, each bound in a different color. Collectively they are known as the Rainbow Series.

The books are officially retired: the TCSEC was cancelled in 2002 [2] and superseded by the Common Criteria, and no product is evaluated against it today. Yet much of the vocabulary that security engineers use every day was first defined in these pages: the trusted computing base, the reference monitor, mandatory access control, and covert channels. When we speak today of browser sandboxes, verified kernels, or CPU side-channel attacks, we are using ideas that the Rainbow Series wrote down forty years ago.

This paper is written as a tutorial for readers who want to understand those ideas without first reading several thousand pages of government guidance. It has two parts. Part I explains the eight most influential volumes one at a time: the TCSEC [1], its glossary [3], and the guides on security modeling [4], discretionary access control [5], audit [6], object reuse [7], data remanence [8], and covert channel analysis [9]. Part II covers the remaining volumes. Many of them are short guides to a single requirement or to the evaluation process, and read one by one they repeat a great deal; grouped by theme, they tell a coherent story about authenticating users, connecting systems, building on trusted foundations, surviving failure, protecting the software supply chain, producing assurance, and running and governing trusted systems.

Every section follows the same pattern. It explains the idea in plain terms with an example, summarizes what the original volumes require, and then gives the essential modern developments: what replaced the idea, what grew out of it, and where it still fails. The modern updates are deliberately selective rather than exhaustive. Each section ends with a short boxed summary of the key point.

The remainder of the paper is organized as follows. Section II proposes a reading order. Part I comprises Sections III–IX, and Part II comprises Sections X–XVII. Section XVIII describes what the series did not foresee, Section XIX assesses which ideas endured, and Section XX concludes. Appendix A provides a glossary, and Appendix B indexes every volume of the series against the section of this paper that discusses it.

II. A Reading Path

The Rainbow Series is easiest to learn in a particular order, shown in Fig. 1. The Orange Book comes first, because every other volume elaborates one of its requirements. The two theoretical guides, on security models and on covert channels, come next. The four practical guides on access control, audit, object reuse, and data remanence follow, completing Part I. The thematic guides of Part II can then be read in any order, according to interest, and the final stage moves beyond the series altogether. At each

Manuscript prepared November 2026.

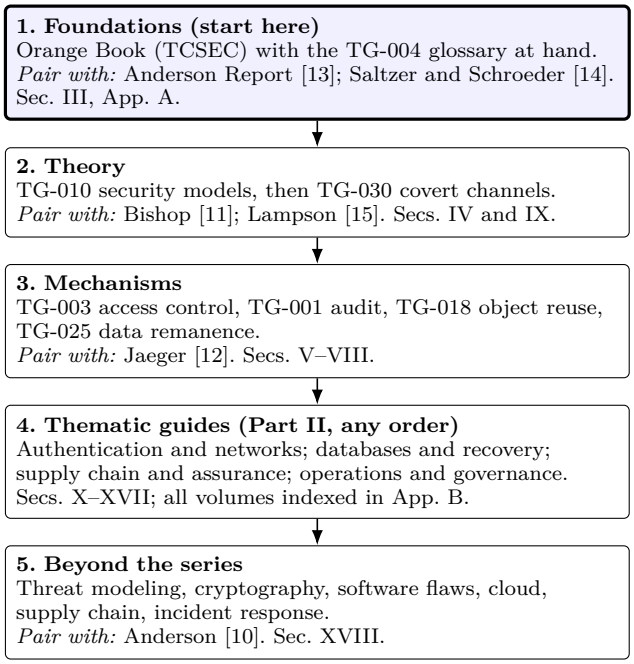


Fig. 1. Recommended reading path through the Rainbow Series and companion sources.

stage it helps to read a modern text alongside the original, so that one meets an idea and its successor together. Anderson [10] is the best choice for breadth, Bishop [11] for formal treatment, and Jaeger [12] for operating system mechanisms.

III. The Orange Book: Defining "Trusted"

The TCSEC [1] set out to answer a practical question for the military: how can one know that a computer will keep information of different classifications apart? Its answer has three parts. Put one small mechanism in charge of security. Ensure that nothing can bypass it. Then collect enough evidence to believe that it works.

A. One Guard at Every Door

The TCSEC describes the world in terms of *subjects*, the active entities such as processes acting on behalf of users, and *objects*, the passive entities such as files, memory segments, and devices. Every attempt by a subject to use an object must pass through a single checkpoint called the *reference monitor*. The idea came from a 1972 Air Force study known as the Anderson Report [13]. A useful picture is a building with exactly one guarded entrance: security depends on there being no side doors, on the guard being impossible to bribe, and on the guard's instructions being short enough to check.

Those three conditions are precisely what the TCSEC requires of the mechanism that implements the reference monitor. It must be 1) tamperproof, 2) always invoked, and 3) small enough to be subject to analysis and tests whose completeness can be assured [1].

Everything on which security depends, whether hardware, firmware, or software, is called the *trusted computing base* (TCB). The TCB matters because of a simple asymmetry: a bug inside the TCB can break security, whereas a bug outside it cannot. The design goal is therefore to keep the TCB as small and as simple as possible.

TABLE I
TCSEC Evaluation Classes [1]

Class	Name	What it adds
D	Minimal protection	Evaluated, but failed every higher class
C1	Discretionary security	Basic owner-controlled access
C2	Controlled access	Individual accountability, audit, clearing of reused storage
B1	Labeled security	Sensitivity labels and mandatory access control
B2	Structured protection	Formal security model, covert storage channel analysis, trusted path
B3	Security domains	Full reference monitor, minimal TCB, trusted recovery
A1	Verified design	Formally verified design, trusted distribution

B. Features Versus Assurance

The most lasting insight of the Orange Book is that trust has two separate dimensions. *Features* are what the system does: logins, access control, audit trails, and labels on classified data. *Assurance* is how confident one can be that those features actually work, based on careful design, testing, documentation, and, at the top, mathematical proof. A system can have impressive features and little assurance, much as a door can have an elaborate lock that no one has ever tested.

The TCSEC combined both dimensions into a ladder of seven classes in four divisions, summarized in Table I. Each rung adds features and assurance to the one below it.

Few products reached the top. Multics was evaluated at B2 in 1985 [16], and Honeywell's SCOMP and the GEMSOS kernel were rated A1. Most commercial vendors stopped at C2, as Windows NT 3.5 with Service Pack 3 did in 1995.

C. What Happened Next

Evaluations were slow and expensive. Worse, because features and assurance were bundled together, a vendor could not obtain high assurance without also building military-style classification labels, which commercial customers rarely wanted. In 1999 the international Common Criteria (ISO/IEC 15408) took over and separated the two dimensions: Protection Profiles describe what a product must do, and Evaluation Assurance Levels EAL1 to EAL7 describe how much evidence backs it [17]. The current edition is CC:2022.

The reference monitor itself never went away. It lives on in Linux Security Modules such as SELinux, in the Windows Security Reference Monitor, in hypervisors, and

in every browser sandbox. The ambition of a small, provably correct TCB was also realized: the seL4 microkernel received a machine-checked proof of functional correctness in 2009 [18]. Hardware trusted execution environments, including Intel SGX and TDX, AMD SEV, and Arm TrustZone and CCA, try to shrink the TCB further by removing the operating system from it. Their repeated difficulties with side-channel attacks (Section IX) show why the Orange Book insisted on covert channel analysis. For an honest retrospective on what the evaluation era achieved, see Karger and Schell [16].

Key point. Keep the trusted part small, put a check on every path, and remember that features without evidence are only promises.

IV. Security Models: Writing Security Down

One cannot prove that a system is secure without first stating exactly what "secure" means. TG-010 [4] explains how to do so with a *security model*: a precise, usually mathematical, description of the rules. The TCSEC requires an informal or formal model at B1 and a formal model from B2 upward [1].

A model has three ingredients. There is a *state*, which records who can access what at the moment; there are *rules* for changing the state; and there are *properties* that must hold in every state the system can reach. The designer proves that the rules never violate the properties, and then shows that the real system behaves like the model. TG-010 also gives a warning that is as true now as it was then: a perfect proof of the wrong policy is worthless.

A. Bell–LaPadula: No Reading Up, No Writing Down

The model behind most of the series is that of Bell and LaPadula [19]. It protects secrets in a world of levels, such as Unclassified, Secret, and Top Secret, and rests on two rules. The *simple security property* says that a subject may not read an object above its clearance ("no read up"). The *⋆-property* says that a subject may not write to an object below its current level ("no write down").

The second rule is the subtle one. Suppose a Secret-level user runs a program that contains a hidden Trojan horse. The first rule cannot stop the program from reading the user's Secret files, because the user is entitled to read them. The second rule stops it from copying them into an Unclassified file where an accomplice could read them.

The weak point of the model is its exceptions. Real systems need *trusted subjects* that may break the ⋆-property, for example to declassify a document. Those exceptions are exactly where an analyst, or an attacker, should look first.

B. The Models That Came After

Bell–LaPadula protects confidentiality only, and later models filled the gaps. Biba's model [20] turns its rules upside down to protect *integrity*: no reading down and no writing up, so that untrusted data cannot contaminate trusted data. It can be seen today in Windows Mandatory Integrity Control, where sandboxed browser processes run at Low or Untrusted integrity. Clark and Wilson [21] observed that businesses care less about secrecy than about correctness, and modeled integrity through certified transactions and separation of duty; any bank's approval workflow follows the same pattern. Brewer and Nash [22] formalized the "Chinese Wall" policy of consulting and finance, in which seeing one company's data blocks access to its competitors'.

Role-based access control [23], standardized as ANSI INCITS 359-2004, attaches permissions to roles rather than to people. Attribute-based access control decides from attributes of the user, the resource, and the context. Together they govern almost every enterprise and cloud system. At a deeper level, Goguen and Meseguer's *noninterference* [24] asks whether anything a high-level user does can have any visible effect on what a low-level user sees. This question underpins covert channel theory and the confidentiality proofs of seL4 [18]. Finally, type enforcement, as implemented in SELinux on Linux and Android, places each process in a domain and lists exactly which kinds of object each domain may touch.

Formal methods have also left the laboratory. Amazon Web Services, for example, uses the TLA+ specification language to find design errors in its distributed systems [25].

Key point. Write the rules down precisely and look hardest at the exceptions. Confidentiality and integrity need different models, and the two often pull against each other.

V. Discretionary Access Control: Who Decides Who Gets In

Sharing a document with a colleague is an everyday example of *discretionary access control* (DAC): the owner of an object decides who else may use it. TG-003 [5] compares the ways to implement DAC and identifies its built-in limitation, one that still shapes security today.

A. Lists Versus Keys

TG-003 describes five mechanisms: capabilities, profiles, access control lists, protection bits, and passwords. Two of them matter most, and an analogy makes the difference clear.

An *access control list* (ACL) is attached to the object, like a guest list held at the door. It is easy to see who has access and easy to strike someone off. It is hard, however, to answer the question "what can this particular user reach?", because one would have to inspect every guest list in the building.

A *capability* is attached to the subject, like a key carried in a pocket. Keys are easy to hand on and can be very precise. The cost is that keys can be copied, and taking them back is difficult.

The remaining three mechanisms are simpler and weaker. Protection bits, the familiar owner/group/other

permissions of Unix, are fast but coarse. Per-user profiles become unmanageable as systems grow. Passwords on objects tend to leak and record nothing about who used them.

B. Three Hard Questions

Whatever the mechanism, TG-003 argues that every design must answer three questions. The first is *propagation*: if I share a file with you, may you share it onward? The second is *revocation*: when I withdraw access, does it stop immediately, and does it also undo any onward sharing? The third is the *Trojan horse* problem. Any program a user runs acts with that user's rights, so a malicious program can legitimately copy the user's private file to a public place. DAC cannot prevent this; only mandatory controls can. This is why the Orange Book requires mandatory access control from B1 upward [1].

C. The Same Problems, Everywhere

These mechanisms are all around us: Unix and Windows file permissions, cloud storage bucket policies, and "anyone with the link" sharing. Capabilities have made a strong comeback as OAuth access tokens, pre-signed URLs, file descriptors, and the handle-based designs of FreeBSD's Capsicum and Google's Fuchsia.

Hardy's "confused deputy" [26] names a related trap: a privileged program tricked into using its own authority on an attacker's behalf. Cross-site request forgery and server-side request forgery are its modern descendants. The Trojan horse problem, meanwhile, has grown into today's software supply-chain problem, in which a malicious package runs with the full rights of the developer who installed it. Sandboxes and permission prompts on phones and in browsers are the modern response.

Key point. Lists say who can reach a thing; keys say what a program can reach. Plan revocation from the start. Owner-controlled access protects users from one another, not from the programs they run.

VI. Audit: Keeping a Record

Access control stops what it can; audit ensures that everything else leaves a trace. TG-001 [6], the Tan Book, treats the audit mechanism as part of the TCB itself, for a simple reason: a log that an attacker can edit proves nothing.

A. What to Write Down

From C2 upward, the system must record logins and other uses of identification and authentication, the creation and deletion of objects, actions by operators and administrators, and other security-relevant events such as permission changes. Each record answers the same questions: when the event happened, who caused it, what kind of event it was, and whether it succeeded. Login events also note where the request came from, object events name the object, and from B1 the object's classification label is recorded as well [1], [6].

The audit trail must itself be protected. Only authorized administrators may read it, no one may modify it, and administrators need tools to search it by user, object, or event. Higher classes go further. At B2 the system must audit events that may exercise covert storage channels, including all such channels faster than ten bits per second. At B3 it must recognize an accumulation of events that signals an imminent violation, such as a run of failed logins, notify the security administrator immediately, and take the least disruptive action that ends the event [6].

B. Logging Today

The requirements have changed little; the scale has changed enormously. Linux auditd, the Windows Security Event Log and Sysmon, and cloud services such as AWS CloudTrail all do what TG-001 describes. What is new is what happens afterward. Logs are shipped off the machine to a security information and event management (SIEM) system, where detection rules, often mapped to the MITRE ATT&CK catalogue of attacker techniques, search for signs of trouble.

Tamper resistance has improved through hash-chained or signed logs and write-once storage. New concerns have appeared too: logs frequently contain personal data covered by privacy law, attackers inject misleading entries, and clearing logs is now a standard step in an intrusion. NIST SP 800-92 [27] remains the principal U.S. guide to log management, and a revision is in draft.

Key point. Record who did what, to what, when, and whether it worked. Send logs where the attacker cannot reach them, and make sure someone reads them.

VII. Object Reuse: Leftovers in Memory

Imagine checking into a hotel room and finding the previous guest's papers in the desk drawer. *Object reuse* is the same problem inside a computer: when the system hands a piece of storage to a new user, none of the previous user's data may remain readable. The rule applies from C2 upward [1], and TG-018 [7] is one of the shortest books in the series. It nevertheless describes one of the most frequently broken rules in computing.

The remedy sounds easy. When a resource is reallocated, revoke the old owner's access and wipe the contents, or block reads until the new owner has written something. The hard part is finding every resource. TG-018 lists memory pages, disk blocks, registers, buffers, and caches, together with easily overlooked corners such as the unused end of a file's last disk block and swap space.

A. Still Breaking Forty Years Later

The rule has not changed, but the places where it fails have multiplied. Kernels have repeatedly leaked uninitialized memory to user programs, which is why Linux now offers the `init_on_alloc` and `init_on_free` options to zero memory automatically. In 2019 the Microarchitectural Data Sampling (MDS) attacks recovered stale data

left in the processor's internal fill buffers, store buffers, and load ports [28]. In 2024 LeftoverLocals showed that one program could read another's leftover data in GPU local memory [29]. Cloud providers, for their part, must clear or encrypt every disk volume before giving it to the next customer. Heartbleed (2014), a missing length check in OpenSSL that let attackers read leftover server memory, private keys included [30], is a close cousin of the same failure.

Key point. Anything handed from one user to another needs a wipe in between, including hardware not usually thought of as storage. In one's own code: initialize before use and clear secrets after use.

VIII. Data Remanence: When Delete Does Not Delete

Pressing "delete" usually removes a file's name, not its contents. TG-025 [8], the Forest Green Book, asks what it takes to get rid of data for real. Its answer is three levels, matched to how determined and well equipped the person trying to recover the data is.

Clearing protects against someone using ordinary system tools and software, which the book calls a "keyboard attack"; a single overwrite of all addressable storage is enough. *Purging* protects against a laboratory with specialized equipment. The DoD approved overwriting and degaussing (erasing magnetic media with a strong magnetic field) for purging, and required degaussing wherever any part of the media could not be overwritten. *Destruction* means physical ruin: disintegration, incineration, pulverization, shredding, or melting. The right level depends on how sensitive the data is and on where the media goes next, whether back into the same office, to a less secure use, or out of the organization's control altogether [8].

A. What Changed With Flash

These three terms survive almost unchanged in NIST SP 800-88, whose latest revision (Rev. 2) appeared in September 2025 [31] and defers media-specific techniques to IEEE 2883-2022 [32]. For practical procedures, the modern documents should be used rather than TG-025.

The largest change is flash storage. Solid-state drives and phones spread writes across spare blocks behind the scenes, so overwriting the visible drive misses hidden copies of the data, as Wei *et al.* demonstrated experimentally [33]. The correct tools now are the drive's own sanitize commands or *cryptographic erase*: if all data was encrypted under a key held inside the drive, destroying the key renders the data unreadable. Full-disk encryption therefore turns secure disposal into a matter of seconds.

Two further lessons follow. First, the famous 35-pass Gutmann overwrite was designed for the disk encodings of the 1990s; a single pass suffices on a modern hard disk. Second, data lingers in more places than disks: printers and copiers, phones, cloud snapshots and backups, and even DRAM, which retains its contents for a short time after power is removed, as the "cold boot" attack showed [34]. In the cloud, customers cannot sanitize the hardware themselves, so the practical equivalent is to encrypt under keys they control and destroy those keys when finished.

Key point. Match the method to the threat. On SSDs and phones, use built-in sanitize or cryptographic erase rather than overwriting tools. Encrypt from the first day and disposal becomes easy.

IX. Covert Channels: Leaks Through the Walls

Suppose every access check in a system is perfect. Could a secret still escape? TG-030 [9], the Light Pink Book, answers yes. Two programs that are forbidden to communicate can still signal each other through anything they share. Of all the Rainbow Series volumes, this is the one that has aged best.

A. Two Kinds of Hidden Channel

Lampson first described the problem in 1973 as the "confinement problem" [15]. TG-030 calls these hidden routes *covert channels* and sorts them into two kinds.

A *storage channel* uses shared state. A high-level sender might fill the disk, create a file, or hold a lock, while a low-level receiver checks whether the disk is full, whether the file exists, or whether the lock is free. Each check reveals one bit, and repeating the procedure transmits an entire message.

A *timing channel* uses time instead. The sender alternately keeps the processor busy or leaves it idle, and the receiver measures how long its own work takes. Slow means one, fast means zero.

B. Finding, Measuring, and Handling Channels

To find channels systematically, TG-030 recommends methods such as Kemmerer's Shared Resource Matrix [35]. One lists every shared attribute in the system and every operation that reads or modifies it; any attribute that one security level can modify and another can observe is a candidate channel. Information flow and noninterference analyses complement the matrix.

Each channel's bandwidth is then estimated, because a very slow leak may be tolerable while a fast one is not. The TCSEC gives rough guidance. A channel faster than 100 bits per second counts as high bandwidth, roughly the speed of a computer terminal of the period. Channels slower than 1 bit per second are acceptable in most environments. Wherever possible, the system should be able to audit any channel that might exceed one bit every ten seconds [1]. Channels can be eliminated by partitioning the shared resource, slowed by adding noise or coarsening clocks, or monitored through audit.

C. From Covert Channels to Side Channels

Today the same science goes by a broader name: *side channels*. A covert channel has a willing sender; a side channel leaks from a victim who has no idea it is transmitting. The discoveries came steadily. Kocher showed that

the time taken to perform cryptographic operations can reveal secret keys [36]. Ristenpart *et al.* showed that an attacker could rent a cloud server on the same physical machine as a target and observe it through shared caches [37]. Flush+Reload turned shared CPU caches into fast, precise leaks across processes and virtual machines [38], and Rowhammer showed that repeatedly accessing one row of memory can flip bits in neighboring rows, breaking memory isolation altogether [39]. Spectre [40] and Meltdown [41] revealed that speculative execution leaks data through cache timing, opening a whole family of processor flaws. Hertzbleed showed that changes in a processor's clock frequency could leak secrets even to a remote attacker [42].

The modern defenses echo TG-030's toolkit. Cryptographic code is written to take the same time regardless of the secret. Caches are partitioned, for example with Intel Cache Allocation Technology. Untrusted workloads are kept off shared CPU cores through core scheduling or by disabling simultaneous multithreading. Browsers coarsen their timers and isolate websites from one another. And processor vendors issue a steady stream of microcode updates.

Key point. Correct access control is not enough. Anything shared, whether cache, processor time, power, or memory, can leak. Today's cache channels run millions of times faster than the Orange Book's thresholds.

X. Identification and Authentication

Every control described in Part I assumes that the system knows who it is talking to. Discretionary access control needs to know who owns a file, mandatory access control needs to know a user's clearance, and audit needs to attribute each event to a person. Two volumes address this "first line of defense": the Green Book on passwords [43] and the Light Blue Book on identification and authentication in general [44].

A. The Green Book: Passwords by Calculation

The *Department of Defense Password Management Guideline* (CSC-STD-002-85, April 1985) [43] is remembered today mainly for rules that have since been reversed, but its reasoning is more interesting than its rules. It advocates three features: users should be able to change their own passwords; passwords should be *machine-generated* rather than user-chosen, because "many user-created passwords are particularly easy to guess"; and the system should tell users directly when they last logged in, so that they can spot misuse.

Its most instructive idea is that password length should be *calculated from the threat*, not fixed by decree. One chooses an acceptable probability P that a password will be guessed during its lifetime, estimates the number of guesses G an attacker can make in that time, and requires a password space of at least $S = G/P$. With a 300-baud line, a six-month lifetime, and $P = 10^{-6}$, the book derives a length of about nine characters from a 26-letter alphabet. Passwords should have a maximum lifetime of no more than one year, be stored only in one-way encrypted form ("there is no need to be able to decrypt passwords"), never be echoed to the screen, and be protected by login rate limits of between one attempt per second and one per minute.

B. The Light Blue Book: Three Kinds of Evidence

TG-017 (1991) [44] generalizes the discussion beyond passwords. It sorts authentication evidence into the three types still taught today: *something you know* (type 1), *something you have* (type 2), and *something you are* (type 3), and it argues that combinations such as "type 12" give more assurance than any single type. It traces how the requirements grow with the TCSEC classes: group identity suffices at C1, individual identity and audit of logins are required from C2, the login establishes the user's security level for mandatory access control from B1, and from B2 a *trusted path* guarantees that the user is talking to the genuine login program rather than a convincing imitation.

The guide is also prescient about attacks. It warns that even the security officer should not be able to read plaintext passwords, recommends genuinely one-way transformations, and describes the "catalog attack", in which an attacker hashes a dictionary of likely passwords and compares the results with stolen hashes, citing the 1988 Internet worm. It notes that networks give intruders many chances to intercept authentication data and that one-time passwords help. It also rejects "personal knowledge" questions, such as a grandmother's maiden name, because the answers are few and often public.

C. Modern Practice

NIST SP 800-63B-4 (2025) [45] keeps the Green Book's goals but reverses several of its rules. Single-factor passwords must now be at least 15 characters long, and verifiers shall not impose composition rules or require periodic changes, though they must force a change when there is evidence of compromise. New passwords must be checked against a blocklist of common and compromised passwords, failed attempts are capped at 100 per authenticator, and stored passwords must be salted and processed with a deliberately costly hashing scheme, which is the modern form of the Green Book's one-way encryption. The one major Green Book idea that has not survived is machine-generated passwords: modern guidance instead accepts user choice and filters out bad choices.

TG-017's three types map directly onto today's *authentication factors*. Current guidance requires that verifiers offer at least one *phishing-resistant* option at the intermediate assurance level, and states plainly that passwords and one-time codes are not phishing-resistant [45]. The leading phishing-resistant mechanism is the passkey, built on the W3C Web Authentication standard [46], [47] and FIDO specifications [48]: a public-key credential bound to a single website and unlocked with the device's PIN

or biometric. In TG-017's terms it is a type 2 credential activated by type 1 or type 3 evidence.

TG-017's worry about interception on networks was answered by protocols that never send the secret at all. Kerberos [49] issues tickets from a trusted key distribution center, and the web's single sign-on systems are built on OAuth 2.0 [50] for delegated authorization and OpenID Connect [51], "an identity layer on top of" OAuth.

Key point. Size defenses to the threat rather than to habit, keep authentication secrets unreadable even to administrators, and combine factors. Today that means long passwords screened against breach lists, no forced rotation, and phishing-resistant passkeys wherever possible.

XI. Networks

The Orange Book describes a single computer. As soon as computers were connected, a new question arose: can a network as a whole be trusted, and how should its security be divided among its parts? The Red Book [52] and its companion environments guideline [53] are the series' answer.

A. The Red Book: A Network as One System

The *Trusted Network Interpretation* (TNI, NCSC-TG-005, July 1987) [52] has two parts. Part I reinterprets the TCSEC classes for networks "ranging from isolated local area networks to wide-area internetwork systems". Its central idea is the *network trusted computing base* (NTCB): one conceptual reference monitor for the whole network, *partitioned* among the components that make it up. A gateway might enforce mandatory access control, a host discretionary control, and a log server audit. No single component need be complete, but the sum of the component policies must be shown to enforce the overall network policy. To make this workable, the TNI classifies components by which of four policy functions they support: mandatory (M), discretionary (D), identification and authentication (I), and audit (A).

Part II adds services that a standalone computer never needed: *communications integrity* (authentication, message integrity, non-repudiation), *denial of service* protection (continuity of operations and network management), and *compromise protection* (data and traffic-flow confidentiality, selective routing). Unlike Part I, these are rated qualitatively, as none, minimum, fair, or good. The TNI defines denial of service in operational terms: it exists "whenever the throughput falls below a pre-established threshold, or access to a remote entity is unavailable".

The TNI also identified the *cascading problem*, which remains a useful way to think about interconnection. Suppose system A is accredited for Secret and Top Secret users and system B for Confidential and Secret users. Each is acceptable on its own, but connecting them creates a path along which an attacker could move Top Secret data down to Confidential users, a range greater than either system was ever trusted to separate.

B. Choosing Protection by Risk

The *TNI Environments Guideline* (NCSC-TG-011, 1990) [53] tells accreditors how much protection a given network needs. It uses a simple *risk index*: the difference between the most sensitive data on the network and the clearance of the least-cleared user. When every user is cleared for the most sensitive data, the risk index is zero: such a network requires C2 if it runs in system-high mode, and has no formal minimum if it is dedicated, although the guideline still recommends C2 for integrity and availability. A gap of one level requires B1, two requires B2, three requires B3, and four requires A1. Larger gaps were declared "beyond the state of current computer security technology". For the Part II services, the guideline computes risk per device, based on who could physically reach it, so a long-haul line or a wireless link that uncleared people can tap demands stronger protection. The same risk-index method appears for standalone systems in the Yellow Books (Section XVII).

C. Modern Practice

The modern network has abandoned the TNI's assumption that a protected perimeter can be trusted. Zero trust architecture [54] grants "no implicit trust ... to assets or user accounts based solely on their physical or network location", secures all communication regardless of location, and grants access per session with least privilege. Its policy decision and enforcement points act rather like distributed reference monitors, a loose echo of the NTCB partitions, and micro-segmentation shrinks the "implicit trust zones" within which the cascading problem can arise.

Several Part II services are now provided by default. TLS 1.3 [55] authenticates the server, protects integrity and confidentiality, removes static key exchange so that every session has forward secrecy, and encrypts nearly the whole handshake. IPsec [56] provides comparable protection at the network layer, and its gateway-to-gateway tunnels resemble the TNI's model of interconnecting separately accredited enclaves. In our assessment, the TNI's lasting contributions are the idea of composing a system's security from the evaluated security of its parts, and the cascading problem, which today goes by the name of lateral movement.

Key point. A network's security is the composition of its parts' security, and connecting two individually acceptable systems can create an unacceptable one. Modern networks assume the perimeter has already failed and authenticate every connection.

XII. Building on a Trusted Base: Databases, UNIX, and Add-On Products

The Orange Book evaluates a whole system from the hardware up. Real products, however, are layered: a database runs on an operating system, an access control package is bolted onto an existing machine, and a UNIX vendor adds features to a familiar kernel. Three volumes ask how trust carries across such layers, and all three reach a cautious answer.

A. The Trusted Database Interpretation

The *Trusted Database Management System Interpretation* (TDI, NCSC-TG-021, April 1991) [57] begins from a historical observation: for years it was believed that "one could not credibly implement a multilevel secure database management system (DBMS) on top of an untrusted operating system". Once evaluated trusted operating systems existed, a secure database could be built on one.

To make that possible, the TDI introduces the *TCB subset*. A TCB subset is defined exactly like a reference monitor (it mediates every access, is tamper resistant, and is small enough to analyze) except that it need not reach down to the hardware; instead it relies on "more primitive TCB subsets" beneath it. A database's TCB subset can thus sit on the operating system's. Under six stated conditions, such as each subset occupying its own protected domain and the lower subsets supporting the upper ones' arguments, the TDI permits *evaluation by parts*: an already-evaluated operating system need not be re-evaluated when a new database is placed on top. It adds a warning that applies to every layered system: "it is not currently possible to determine the trust characteristics of a system on the basis of an arbitrary collection of subparts."

The TDI leaves the designer free to choose what gets labeled, whether a table, a row, or a field, but requires that metadata such as indices, schemas, and logs be labeled too. Notably, it did *not* solve the hardest database problems. Content- and context-dependent controls were declared "research topics", and the problems were taken up in a five-volume 1996 companion series [58], [59]. Two of them are worth understanding. *Inference* is the deduction of secret facts from visible ones, for example working out a classified destination from unclassified fuel and cargo records; it is dangerous precisely because "the leakage of information is not a result of the penetration of a security mechanism" [58]. *Polyinstantiation* is a deliberate response: the database stores several versions of the same record at different levels, so that a low-level user sees an innocuous cover story rather than a suspicious refusal. Its cost is stated bluntly: "integrity across levels cannot be enforced if multilevel confidentiality is strictly enforced" [59].

B. TRUSIX: Designing ACLs for UNIX

The Trusted UNIX Working Group was formed in 1987 to guide vendors building B3 UNIX systems. Its first report (NCSC-TG-020-A, 1989) [60] is a short case study in engineering trade-offs. UNIX already had owner/group/other permission bits, and access control lists had to be added without breaking existing programs. TRUSIX weighed six designs and chose to let the existing group permission bits act as a *mask*, an upper bound on whatever the list grants. That choice preserved a vital property: running `chmod` to remove all permissions still "unequivocally" revokes access from everyone. The group also chose a fixed, system-defined order for checking entries (owner, named users, groups, others) rather than letting users order them, to avoid surprises.

C. Rating Add-On Subsystems

In the 1980s many organizations bought add-on packages to improve the security of existing systems. The *Computer Security Subsystem Interpretation* (NCSC-TG-009, 1988) [61] rated four kinds separately (discretionary access control, object reuse, identification and authentication, and audit), each on its own small scale within the D division, because no subsystem meets all the requirements of a full C1 system. It recognized that the functions depend on one another, since audit needs identification and identification data needs protection, and it required developers to show that a subsystem could be integrated. Like the TDI, it warned that "an overall system rating ... cannot be inferred from the application of one or more separately-rated subsystems."

D. Modern Practice

Row-level security is now a standard database feature. PostgreSQL added it in version 9.5 [62]; with it enabled and no policy defined, access is denied by default, and policies are checked before any user-supplied conditions [63]. Microsoft SQL Server [64] and Oracle Label Security [65] offer similar controls, the latter with explicit sensitivity labels in the TDI tradition. Even here the TDI's tensions resurface: PostgreSQL's documentation warns that integrity checks such as unique keys always bypass row security and can create "covert channel" leaks [63]. The inference problem has its principled modern answer in *differential privacy* [66], which the U.S. Census Bureau adopted for the 2020 census [67], while SQL injection [68] shows that an application can still bypass the database's reference monitor entirely.

TRUSIX's design lives on in Linux, whose POSIX ACLs use exactly the group-mask scheme it chose [69]; NFSv4 ACLs [70], by contrast, adopt the user-ordered allow/deny design that TRUSIX rejected. Add-on security has become the Linux Security Modules framework [71], which lets SELinux, AppArmor, and others plug into defined kernel hooks, and the Common Criteria formalized TG-009's idea through *PP-Modules*, which add requirements to a base protection profile rather than standing alone [72].

Key point. Trust can be layered, but only under explicit conditions; it cannot be assembled from arbitrary parts. In databases, confidentiality and integrity collide, and inference can leak secrets without breaking any rule.

XIII. Recovery and Resilience

Every system eventually crashes. The question the *Guide to Understanding Trusted Recovery* (NCSC-TG-022, 1991) [73] asks is whether it comes back *securely*. The TCSEC requirement, which applies only at B3 and A1, is that "after an ADP system failure or other discontinuity, recovery without a protection compromise is obtained" [1].

A. Failures and Secure States

The guide sorts failures into four kinds: a single operation failing (for example, running out of table space halfway through creating a file), a failure of the TCB itself, which always crashes the system, a failure of storage media that may corrupt security data such as labels, and a *discontinuity* caused by people, such as switching off power or ignoring a warning that the audit trail is nearly full. The key concept is the *secure state*: a state in which all the invariants of the security model hold. A trusted system must either stay in secure states or, after a failure, detect that it is in an insecure one and rebuild a secure one before letting users back in.

There are three ways back. A *reboot* is a controlled restart when data structures are known to be consistent. An *emergency restart* performs recovery automatically in a maintenance mode. A *cold start* is needed when automatic recovery fails, and requires intervention by administrators; at B3, those recovery functions must be restricted to staff with security responsibilities. To make recovery tractable, the guide recommends that security-critical updates, such as changes to the password file or an access control list, be *atomic*: they either complete fully or not at all, implemented by keeping a shadow copy or a log of the update.

B. Modern Practice

The same ideas now protect the lowest layers of the machine. NIST's platform firmware resiliency guidelines [74] organize firmware security around three principles: *protect* against unauthorized changes, *detect* them, and *recover* rapidly and securely, with recovery classified as fully automated, partially automated, or manual, a close match to TG-022's emergency restart and cold start. UEFI Secure Boot [75] refuses to start any boot image not signed by an authorized key, and TPM-based measured boot [76] records what was started so that a remote party can detect an insecure start state. At the application level, write-ahead logging, formalized in the ARIES algorithm [77] the year after TG-022, is the standard way to make updates atomic, and operating systems that keep two copies of the system image and fall back to the old one after a failed update apply TG-022's shadowing idea to the entire system.

Key point. Security must survive failure: define what a secure state is, make critical updates atomic, and restrict manual recovery to trusted staff. Today the same principles protect firmware and the boot process.

XIV. The Software Supply Chain

A system is only as trustworthy as the process that built it and the route by which it reached its user. Two volumes address these concerns, configuration management [78] and trusted distribution [79], and both read today like early drafts of software supply-chain security.

A. Configuration Management

A Guide to Understanding Configuration Management in Trusted Systems (NCSC-TG-006, 1988) [78] defines configuration management (CM) as four tasks: *identification* of every item that makes up the system, *control* of changes to those items, *status accounting* that records what changed, and *auditing* that verifies each change is correct and, for trusted systems, "consistent with the security policy of the system". From B2 upward, every change to the TCB must pass through this process, covering not just source code but design documents, test material, and the running object code, with "a consistent mapping among all documentation and code".

Two of its requirements are strikingly modern. B2 systems must provide tools to regenerate the TCB from source and to compare "a newly generated TCB version with the previous version in order to ascertain that only the intended changes have been made". At A1, the CM tools themselves must be "maintained under strict configuration control", and the master copies of everything used to generate the TCB must be physically, technically, and procedurally protected. The second rule addresses the problem Thompson later made famous: a compromised compiler can insert a backdoor that appears in no source code [80].

B. Trusted Distribution

A Guide to Understanding Trusted Distribution in Trusted Systems (NCSC-TG-008, 1988) [79] covers the journey from vendor to customer. Required only at A1, the class at which "the threat of subversion is addressed", it asks for assurance that what arrives at the site is exactly what the master copy says it should be, and it names two threats: tampering in transit, and an impostor shipping a fake "update". The guide reviews the methods of its day: tamper-evident packaging and seals, bonded couriers, message authentication codes of the kind banks use, encryption, and acceptance testing at the site. On checksums it is especially perceptive: a 16-bit checksum can be defeated by padding a modified TCB until the checksum matches, so A1 systems need "cryptographically protected checksums". It also recognizes the limit of distribution controls: if the master copy was tampered with during development, "the damage has already been done".

C. Modern Practice

Two recent incidents show why these requirements matter. In the SolarWinds compromise disclosed in December 2020, trojanized updates to a network management product were digitally signed and posted to the vendor's own update site [81]; the malicious code had been inserted by an implant on the vendor's *build server* during compilation [82]. Distribution controls worked perfectly and delivered a signed backdoor; what failed was the protection of the build that TG-006 requires. In 2024 a backdoor was discovered in the xz compression library, hidden in release tarballs that differed from the project's public repository

and apparently inserted by a compromised or malicious maintainer [83], [84], [85]. That is precisely a break in TG-006's consistent mapping between source and released code, carried out by the insider threat TG-008 associates with A1.

The modern defenses map closely onto the old requirements. The SLSA framework [86], [87] grades build integrity in levels, from merely recording provenance to hardened builds that resist tampering "by insider threats, compromised credentials, or other tenants", and its newer source track grades version control and code review much as TG-006 graded CM by class. *Reproducible builds*, in which "any party can recreate bit-by-bit identical copies" of an artifact from its source [88], are a decentralized version of TG-006's regenerate-and-compare requirement. Sigstore [89] signs artifacts with short-lived certificates tied to an identity and records every signature in a public transparency log. Following Executive Order 14028 [90], U.S. policy calls for a software bill of materials listing every component [91], and NIST's Secure Software Development Framework [92] asks developers to protect all forms of code from tampering and to publish integrity information for each release. The Common Criteria's life-cycle class [17] formalizes the same ideas, and its newest component asks developers to record exactly which development artifacts and tools produced the evaluated product.

Key point. Control and record every change, be able to rebuild the product and prove that only intended changes went in, protect the build tools as carefully as the product, and verify what arrives. A signed backdoor is still a backdoor.

XV. The Assurance Process

The Orange Book distinguishes features from assurance (Section III). Several volumes describe how assurance was actually produced: by documenting the design, testing it, proving it, keeping a rating current across new versions, and working through the evaluation itself.

A. Design Documentation

A Guide to Understanding Design Documentation in Trusted Systems (NCSC-TG-007, 1988) [93] lists what the design documents must show at each class, and the list grows steadily. At C1 they must explain the system's philosophy of protection and how the TCB protects itself. B1 adds a security policy model. B2 requires a formal model and a *descriptive top-level specification* (DTLS) of the TCB interface, with an explanation of why the reference monitor cannot be tampered with or bypassed and an analysis of covert storage channels. A1 adds a *formal top-level specification* (FTLS) and shows that it is consistent with the model. The effort was considerable: at the higher classes, design documentation was "roughly comparable in bulk to the source listings of the overall system", and could take "up to several man years".

B. Security Testing

A Guide to Understanding Security Testing and Test Documentation in Trusted Systems (NCSC-TG-023, 1993) [94] explains how testing intensifies. Functional security testing is required at every class, and "penetration testing on B2, B3, and A1 class systems". Testers rank potential vulnerabilities into a hierarchy, "with the most critical and most easily executed attacks attempted first", using the flaw-hypothesis method of guessing where the design is weak and then trying to exploit it. Coverage also grows with the class, from at least five test conditions for each TCB primitive at the lowest classes to at least twenty-five, derived from the formal specification, at A1. The guide distinguishes black-box testing of the interface, white-box testing from the design, and an intermediate gray-box approach required for the whole kernel at B3.

C. Formal Verification Tools

At A1, verification evidence had to be produced with a tool on the NCSC's *Endorsed Tools List*. The *Guidelines for Formal Verification Systems* (NCSC-TG-014, 1989) [95] set the requirements for such tools. The most important is that "proof strategies checked by an independent, trustworthy proof checker shall ensure that only sound proof steps were employed", which is the principle of a small, trusted checking kernel used by modern proof assistants. The list itself was short: it contained only the Formal Development Methodology with its Ina Jo language and the Gypsy Verification Environment, the latter used for SCOMP [96]. The list entry also noted Gypsy's "lack of a formal basis and formal semantic definition" as a serious shortcoming.

D. Keeping a Rating Current

A rating applied only to the exact version evaluated: "once a vendor releases a new version of a product that has received an EPL rating, the new version is not an evaluated product". The Rating Maintenance Phase (RAMP) program [97] let vendors extend a rating to new versions by training a *vendor security analyst*, following an approved rating maintenance plan, analyzing the security impact of every change, and presenting the result to a review board. Crucially, "the burden of proof for RAMP efforts lies with ... the vendor instead of with an evaluation team".

E. The Evaluation Itself

Two volumes guided vendors through the process. *Trusted Product Evaluations: A Guide for Vendors* (NCSC-TG-002) [98] described the phases from an initial proposal review, through design analysis and hands-on functional and penetration testing, to a final evaluation report and an entry on the Evaluated Products List. It cautioned that a listing "does not constitute a general or overall endorsement" and that "testing is not sufficient to guarantee the absence of flaws". The *Trusted Product Evaluation Questionnaire* (NCSC-TG-019) [99] was the

evaluators' structured data-gathering tool, organized class by class across fourteen subject areas from subjects and objects to testing and modeling.

F. Modern Practice

Each of these activities has a direct counterpart in the Common Criteria [17]: design documentation in the development class (with functional specifications and designs up to fully formal ones), testing in the tests class (coverage, depth, and independent testing), and penetration testing in vulnerability analysis, graded up to an "advanced methodical" analysis. RAMP's successor is *assurance continuity* [100]: the developer writes an impact analysis of each change, minor changes are handled by maintenance, and major changes require re-evaluation.

Testing has been transformed by automation. Hand-derived test conditions have been joined by large-scale fuzzing; Google's OSS-Fuzz service reports that, as of August 2023, it had helped identify and fix over 10,000 vulnerabilities across 1,000 open-source projects [101]. Formal verification has matured even further. seL4 was verified in the Isabelle/HOL proof assistant [18], whose small checking kernel is exactly the "independent, trustworthy proof checker" TG-014 required, and modern tools such as the Rocq Prover (formerly Coq) [102], Lean [103], and TLA+ [25] rest on formal semantics, the very property whose absence the endorsed tools list criticized in Gypsy. There is, however, no longer any government list of endorsed tools.

Key point. Assurance is evidence: designs explained in writing, tests that grow with the stakes, proofs checked by a small trusted kernel, and a process for keeping that evidence current as the product changes.

XVI. Operating a Trusted System

A well-designed system can still be run insecurely. Five volumes deal with the people who operate trusted systems: how their powers are divided, what they must be told, and who checks the result.

A. No All-Powerful Role

The *Guide to Understanding Trusted Facility Management* (NCSC-TG-015, 1989) [104] explains a requirement that begins at B2: the system must support separate *operator* and *administrator* functions. At B3 it goes further: administrators may perform security functions only "after taking a distinct auditable action to assume the Security Administrator role", and every action of security personnel must be audited.

The guide recommends a finer division into six roles: security administrator, secure operator, account administrator, auditor, operator, and system programmer. It singles out the separation of the *auditor* from the security administrator as "an important application of the separation of privilege and least privilege principles". Without it, "the entire security function would become the responsibility of a single, unaccountable individual" who could misuse authority "and then use Auditor functions to erase any evidence". The stated goal is memorable: a system "whose mode of operation does not include an all-powerful role". Interestingly, the guide admits that the TCSEC itself does not require that particular separation.

B. Manuals for Administrators and Users

Two volumes describe the documents that must accompany a trusted system. The *Trusted Facility Manual* (TFM), covered by NCSC-TG-016 (1992) [105] and required at every class, tells administrators how to install, configure, and operate the system securely, and how to avoid "pitfalls and improper use of the administrative functions". The guide's reasoning is that "the security damage that administrative users can cause by careless use, or deliberate misuse, of administrative authority is significantly larger than that caused by ordinary users". A good TFM therefore states its assumptions explicitly, including physical security, and gives examples of correct use as well as warnings.

The *Security Features User's Guide* (SFUG), covered by NCSC-TG-026 (1991) [106], is the ordinary user's counterpart: a single document that describes the protection mechanisms available to every user and how to use them. The guide advises against folding it into the TFM, since that "tends to make the system configuration and vulnerability information available to anyone".

C. The Security Officer and the Assessor

NCSC-TG-027 (1992) [107] defines the *information system security officer* (ISSO), the person responsible day to day for a system's security: checking users' clearances and need-to-know before granting access, reviewing audit trails regularly ("weekly or daily"), reporting incidents, and reporting the system's security status to the accrediting authority. The guide applies separation of duties to the security staff themselves; for example, the security manager above the ISSO "should not participate in the day-to-day operation" of the system.

NCSC-TG-028 (1992) [108] gives a practical method for assessing whether a fielded system actually provides C2 protection, which a 1987 national policy had required within five years. It pulls together nine earlier Rainbow volumes into an eleven-step process that starts, tellingly, with configuration management, so that every later fix is tracked. Its advice is blunt: "by far the easiest and most direct way to attain controlled access protection is to build the system on a product that has been evaluated by NSA and rated C2 or higher", used in its evaluated configuration.

D. Modern Practice

These ideas are now written into the standard control catalogue, NIST SP 800-53 [109]. Its separation-of-duties (AC-5) and least-privilege (AC-6) controls require non-privileged accounts for non-security work and logging of

privileged actions, which is essentially TG-015's B3 rule, and control SA-5 calls for administrator documentation of secure configuration and known vulnerabilities, and user documentation of security functions and responsibilities, which is the TFM and SFUG under new names. In Common Criteria evaluations the same documents are the AGD (guidance) deliverables [17]. Secure configuration guidance today is often produced by third parties, such as the consensus-based CIS Benchmarks [110]. The security officer has grown into a senior role: U.S. federal law now requires each agency to have a senior information security officer whose primary duty is information security [111].

Key point. Divide administrative power so that no single person can both act and erase the evidence, document secure operation for administrators and users separately, and audit the auditors.

XVII. Governance: Risk, Accreditation, and Procurement

The final group of volumes answers management questions. How much protection does a particular system need? Who decides that it is safe to operate? And how can an organization buy a trusted system fairly?

A. The Yellow Books: A Risk Index

CSC-STD-003-85 and its technical rationale, CSC-STD-004-85 (both June 1985) [112], [113], tell an organization which TCSEC class to require. They define a *risk index* as "the disparity between the minimum clearance or authorization of system users and the maximum sensitivity of data processed". Clearances and data are scored on matching scales (for example, Confidential is 2 and Secret is 3), with a deliberate jump from Secret to Top Secret (5) to reflect the step from "serious" to "exceptionally grave" damage. The risk index is the difference between the two.

The index then selects a minimum class. A system where everyone is cleared for all data (risk index 0) needs C2 in system-high mode; an index of 1 requires B1 and 2 requires B2. For higher indices the answer depends on the *environment*: in an *open* environment, where developers are not cleared or applications are not protected from malicious code, an index of 3 requires B3 and 4 requires A1, while a *closed* environment can accept one class lower. Indices of 5 and above, for example uncleared users on a system holding Top Secret data, were considered "beyond the state of current computer security technology". The rationale volume notes that "most systems are in open environments" [113]. The network guideline of Section XI applies the same method to networks.

B. Certification and Accreditation

NCSC-TG-029 (1994) [114] explains three terms that are often confused. *Evaluation* analyzes a product against criteria "without regard to the environment". *Certification* analyzes a particular system against its requirements "in a given environment". *Accreditation* is "the official management authorization to operate a system" at "an acceptable level of risk for which the accrediting authority has formally assumed responsibility". The person who signs, the *designated approving authority* (DAA), personally accepts the remaining risk. Accreditation was granted for a limited time, and DoD policy required reaccreditation every three years "regardless of change". For networks, the guide stresses that the boundaries of each component, and responsibility for certifying each one, must be clearly defined.

C. Buying Trusted Systems

The multi-volume *Guide to Procurement of Trusted Systems* (NCSC-TG-024) [115], [116], [117] helped contracting staff write security requirements into requests for proposals, including ready-made specification text. Its most interesting rules concern fair competition: if fewer than five products on the Evaluated Products List meet the requirement, the request may not insist on an evaluated product, nor may it simply state that "a B2 system is required", because that would exclude unevaluated bidders. Instead it must spell out the TCSEC requirements themselves [116].

D. Modern Practice

The NIST Risk Management Framework [118] replaced certification and accreditation with a seven-step process (prepare, categorize, select, implement, assess, authorize, monitor). The DAA became the *authorizing official*, "the only organizational official who can accept the security and privacy risk", and the fixed three-year cycle can give way to *ongoing authorization* supported by continuous monitoring. The U.S. Department of Defense adopted the framework in 2014, replacing its earlier accreditation process [119].

The Yellow Books' risk index has been replaced by FIPS 199 [120], which categorizes a system by the potential *impact* of a loss of confidentiality, integrity, or availability (low, moderate, or high) instead of the gap between clearances and classifications. Procurement has followed the same path as evaluation. FedRAMP (2011) [121] standardizes security authorization of cloud services so that one assessment can be reused across agencies, and U.S. national security systems must buy commercial products from the NIAP Product Compliant List of Common Criteria-evaluated products [122], the direct descendant of buying from the Evaluated Products List.

Key point. Match protection to risk, make a named official accept the residual risk in writing, and buy from evaluated products without shutting out fair competition. Modern frameworks keep all three ideas but measure risk by impact rather than clearance levels.

XVIII. What the Series Did Not Foresee

The Rainbow Series pictures one organization running one closed computer, or a closed network of them, with trusted administrators, software written in-house, and a

cleared insider with a Trojan horse as the main adversary. Part II showed that several later volumes reached toward networks, supply chains, and governance, but most security work today still happens outside that picture. A modern reader needs to fill three kinds of gap.

A. The World Became Connected

The series predates the public internet and treats cryptography only in passing, yet cryptography now protects almost all data in transit and at rest. Aumasson [123] offers an accessible introduction, and Boneh and Shoup [124] a rigorous and freely available one. The series never met the web or mobile devices, today's dominant platforms; Zalewski [125] and the OWASP Mobile Application Security Verification Standard cover them. Nor did it imagine the cloud, where strangers share hardware and identity replaces the network perimeter; the zero trust response is described in Section XI.

B. The Attacker Changed

The Orange Book assumes that the trusted core is correct. In practice, flaws in software are the main way in, and Dowd *et al.* [126] remain the standard guide to finding them. Memory safety is a large part of the story: about 70% of the vulnerabilities to which Microsoft assigns CVEs [127], and of Chromium's high-severity security bugs [128], are memory-safety bugs. This is why memory-safe languages such as Rust, and capability hardware such as CHERI, now attract so much attention.

The configuration management and trusted distribution guides anticipated part of the supply-chain problem (Section XIV), but they assumed that the vendor wrote the code. Today most code that organizations run comes from thousands of open-source dependencies maintained by strangers, which turns the problem from controlling one vendor's process into tracing an entire ecosystem. Finally, machine learning systems introduce attacks of their own, such as prompt injection and data poisoning [129], [130].

C. The Defender's Job Changed

The series hands the reader a fixed threat. Today each organization must work out its own, a practice called threat modeling [131]. The series also assumes that prevention works, whereas modern security programs plan for breach through detection and incident response [132], [133].

People matter as well. Systems that are hard to use get used insecurely, and phishing routinely defeats access control; Whitten and Tygar's study of PGP [134] launched the field of usable security. Privacy, meaning the protection of personal data rather than state secrets, now has its own law and frameworks. And Anderson's economic analysis [135] shows that incentives, more than technology, often explain why insecure products succeed.

The central idea of the Rainbow Series nevertheless applies to every one of these areas: define the policy, keep the trusted part small, check every path, and gather evidence that the check works.

TABLE II
RAINBOW SERIES CONCEPTS AND THEIR CONTEMPORARY DESCENDANTS

Rainbow Series concept	Contemporary descendant
Reference monitor, minimal TCB	Microkernels, hypervisors, sandboxes, seL4 [18]
Mandatory access control and labels	SELinux, Android application sandbox, Windows integrity levels
Formal models and verification	seL4 proofs, verified compilers, TLA+ in industry [25]
Protected audit	Off-host, tamper-evident logging and SIEM
Object reuse	Memory and buffer clearing in operating systems, CPUs, and GPUs
Clear, purge, destroy	NIST SP 800-88 [31]
Covert channel analysis	Side-channel research and constant-time code
Trusted path	Ctrl+Alt+Del secure attention sequence, OS-drawn permission prompts
Trusted distribution, configuration management	Software supply-chain security (SLSA, Sigstore, reproducible builds)
Authentication types 1, 2, 3	Authentication factors, passkeys [47]
Partitioned network TCB	Zero trust policy enforcement points [54]
TCB subsets, evaluation by parts	Layered evaluation, CC composition and PP-Modules
Trusted recovery	Firmware resiliency, secure and measured boot [74]
Separation of administrative roles	Least privilege and separation of duties (SP 800-53 AC-5, AC-6)
Rating maintenance (RAMP)	Common Criteria assurance continuity
Risk index	FIPS 199 impact categorization [120]
Certification and accreditation	Risk Management Framework authorization [118]
Evaluated Products List	NIAP Product Compliant List, FedRAMP

XIX. Discussion: What Held Up and What Did Not

Forty years on, the ideas of the series have aged far better than the process built around them. Table II maps each core concept to its modern descendant.

Several elements did not last. Tying features to assurance meant that high assurance was unavailable without military-style labels, a limitation the Common Criteria later removed [17]. Evaluations were so slow that products were often obsolete by the time they were rated. Some specific rules have been reversed outright: forced password rotation and machine-generated passwords have been abandoned, and NIST SP 800-63B-4 now states that verifiers shall not require periodic password changes [45]. Fixed reaccreditation cycles are giving way to continuous monitoring, and risk is now measured by the impact of a loss rather than by the gap between clearances and classifications. Finally, the whole series assumes a closed world, with no internet, no third-party code, and no plan for what happens after a breach.

Part II also reveals a theme that runs through the whole series: repeated caution about *composition*. The network interpretation, the database interpretation, and the subsystem interpretation each warn that the trustworthiness of a whole cannot be inferred from an arbitrary collection

of trusted parts. That warning has lost none of its force in an era of microservices, cloud platforms, and deep software supply chains.

XX. Conclusion

The Rainbow Series remains one of the clearest introductions to the architecture of trusted systems. Its central contributions (the reference monitor, the trusted computing base, the distinction between features and assurance, and covert channel analysis) continue to shape system design, even though its evaluation regime and several specific requirements have been superseded. A reader new to the material should begin with the TCSEC [1], continue with the guides on security models [4] and covert channels [9], and then study the four mechanism guides [5], [6], [7], [8], reading each alongside a modern text [10], [11], [12]. The thematic volumes of Part II are best read selectively: the network interpretation [52], the database interpretation [57], and the configuration management guide [78] still repay study in full, while the procedural volumes are now mainly of historical interest, their substance having passed into the Common Criteria and the NIST Risk Management Framework.

For historical depth, the foundational papers repay reading: the Anderson Report [13]; Saltzer and Schroeder's design principles [14]; Lampson's papers on protection [136] and confinement [15]; the Bell–LaPadula exposition [19]; Thompson's demonstration that the TCB includes one's compiler [80]; Karger and Schell's retrospective [16]; and the verification of seL4 [18].

Appendix A
Glossary

Table III lists the terms from the NCSC glossary [3] that recur throughout the series, together with their contemporary equivalents. The glossary also defines the "system high" mode, in which every user is cleared for the highest level of data on the system, and the "dedicated" mode, in which every user is cleared for all of it. Both modes are still used in defense accreditation.

Appendix B
Index of the Volumes

Table IV lists the volumes of the series discussed in this paper, with the section that explains each one. Cover colors vary between printings and between the two main online indexes (the FAS archive and Wikipedia), so a color is given only where they agree; dates are those of the volume's foreword or cover.

References

[1] U.S. Department of Defense, "Department of defense trusted computer system evaluation criteria," U.S. Department of Defense, Tech. Rep. DoD 5200.28-STD, Dec. 1985. [Online]. Available: https://csrc.nist.gov/csrc/media/publications/conference-paper/1998/10/08/proceedings-of-the-21st-nissc-1998/documents/early-cs-papers/dod85.pdf

[2] ——, "Information assurance (IA)," DoD Directive 8500.1, Oct. 2002.

[3] National Computer Security Center, "Glossary of computer security terms," National Computer Security Center, Tech. Rep. NCSC-TG-004, Oct. 1988.

[4] ——, "A guide to understanding security modeling in trusted systems," National Computer Security Center, Tech. Rep. NCSC-TG-010, Oct. 1992.

[5] ——, "A guide to understanding discretionary access control in trusted systems," National Computer Security Center, Tech. Rep. NCSC-TG-003, Sep. 1987. [Online]. Available: https://irp.fas.org/nsa/rainbow/tg003.htm

[6] ——, "A guide to understanding audit in trusted systems," National Computer Security Center, Tech. Rep. NCSC-TG-001, Jun. 1988. [Online]. Available: https://irp.fas.org/nsa/rainbow/tg001.htm

[7] ——, "A guide to understanding object reuse in trusted systems," National Computer Security Center, Tech. Rep. NCSC-TG-018, Jul. 1992.

[8] ——, "A guide to understanding data remanence in automated information systems," National Computer Security Center, Tech. Rep. NCSC-TG-025, Sep. 1991. [Online]. Available: https://irp.fas.org/nsa/rainbow/tg025-2.htm

[9] ——, "A guide to understanding covert channel analysis of trusted systems," National Computer Security Center, Tech. Rep. NCSC-TG-030, Nov. 1993.

[10] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Indianapolis, IN, USA: Wiley, 2020. [Online]. Available: https://www.cl.cam.ac.uk/archive/rja14/book.html

[11] M. Bishop, *Computer Security: Art and Science*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

[12] T. Jaeger, *Operating System Security*. San Rafael, CA, USA: Morgan & Claypool, 2008.

[13] J. P. Anderson, "Computer security technology planning study," USAF Electronic Systems Division, Bedford, MA, USA, Tech. Rep. ESD-TR-73-51, Oct. 1972.

[14] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[15] B. W. Lampson, "A note on the confinement problem," *Commun. ACM*, vol. 16, no. 10, pp. 613–615, Oct. 1973.

[16] P. A. Karger and R. R. Schell, "Thirty years later: Lessons from the Multics security evaluation," in *Proc. 18th Annu. Comput. Security Appl. Conf. (ACSAC)*, 2002, pp. 119–126.

[17] "Common criteria for information technology security evaluation," CC:2022, Release 1, 2022. [Online]. Available: https://www.commoncriteriaportal.org/cc/index.cfm

[18] G. Klein *et al.*, "seL4: Formal verification of an OS kernel," in *Proc. 22nd ACM Symp. Operating Syst. Principles (SOSP)*, Oct. 2009, pp. 207–220.

[19] D. E. Bell and L. J. LaPadula, "Secure computer system: Unified exposition and Multics interpretation," MITRE Corp., Bedford, MA, USA, Tech. Rep. MTR-2997 Rev. 1 (ESD-TR-75-306), Mar. 1976.

[20] K. J. Biba, "Integrity considerations for secure computer systems," MITRE Corp., Bedford, MA, USA, Tech. Rep. MTR-3153, Apr. 1977.

[21] D. D. Clark and D. R. Wilson, "A comparison of commercial and military computer security policies," in *Proc. IEEE Symp. Security Privacy*, Apr. 1987.

[22] D. F. C. Brewer and M. J. Nash, "The Chinese Wall security policy," in *Proc. IEEE Symp. Security Privacy*, 1989, pp. 206–214.

[23] D. F. Ferraiolo and D. R. Kuhn, "Role-based access controls," in *Proc. 15th Nat. Comput. Security Conf.*, Baltimore, MD, USA, Oct. 1992, pp. 554–563.

[24] J. A. Goguen and J. Meseguer, "Security policies and security models," in *Proc. IEEE Symp. Security Privacy*, Apr. 1982.

[25] C. Newcombe *et al.*, "How Amazon Web Services uses formal methods," *Commun. ACM*, vol. 58, no. 4, pp. 66–73, Apr. 2015.

[26] N. Hardy, "The confused deputy: (or why capabilities might have been invented)," *ACM SIGOPS Oper. Syst. Rev.*, vol. 22, no. 4, pp. 36–38, Oct. 1988.

[27] K. Kent and M. Souppaya, "Guide to computer security log management," NIST, Special Publication 800-92, Sep. 2006.

[28] MITRE, "CVE-2018-12130: Microarchitectural fill buffer data sampling (MFBDS)," 2019. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2018-12130

TABLE III
SELECTED TERMS FROM NCSC-TG-004 AND THEIR CONTEMPORARY EQUIVALENTS

Term	Meaning	Contemporary equivalent or usage
Subject	An active entity (process, user session) that requests access	Principal, process, workload identity
Object	A passive entity holding data (file, segment, device)	Resource
Reference monitor	The concept of a mechanism that mediates every access	Policy enforcement point
Reference validation mechanism	The implementation of the reference monitor	Security kernel, LSM hooks
Trusted computing base (TCB)	Everything on which security depends	Same term; related to "attack surface"
Security kernel	The hardware and software core implementing the reference monitor	Microkernel, separation kernel, hypervisor
Security policy	The rules governing who may access what	Policy, authorization model
Discretionary access control	Access decided by the object's owner	File permissions, ACLs, sharing links
Mandatory access control	Access decided by system-wide labels that users cannot change	SELinux, Windows integrity levels, MLS
Sensitivity label	Classification level and categories attached to subjects and objects	Security context, data classification tag
Clearance	The highest level at which a user may work	Same; also an attribute in ABAC
Least privilege	Granting each subject only the access it needs	Same; least-privilege IAM, capabilities
Trusted path	A channel to the TCB that software cannot imitate	Secure attention sequence, OS-drawn prompts
Object reuse	Reallocation of storage without clearing old data	Memory and disk zeroing
Data remanence	Data left behind after deletion	Media sanitization
Covert channel	A path that moves information against the policy	Covert and side channels
Assurance	Grounds for confidence that the system meets its policy	Evaluation, verification, testing
Accreditation	Formal approval to operate a system in a given environment	Authorization to operate (NIST RMF)
Multilevel secure (MLS)	Handles several classification levels at once, separating users not cleared for all of them	Cross-domain solutions

TABLE IV
INDEX OF RAINBOW SERIES VOLUMES DISCUSSED IN THIS PAPER

Volume	Title (abbreviated)	Date	Section
DoD 5200.28-STD	Trusted Computer System Evaluation Criteria (Orange)	Dec. 1985	III
CSC-STD-002-85	Password Management Guideline (Green)	Apr. 1985	X
CSC-STD-003-85	Guidance for Applying the TCSEC in Specific Environments (Light Yellow)	Jun. 1985	XVII
CSC-STD-004-85	Technical Rationale Behind CSC-STD-003-85 (Yellow)	Jun. 1985	XVII
NCSC-TG-001	Audit in Trusted Systems (Tan)	Jun. 1988	VI
NCSC-TG-002	Trusted Product Evaluations: A Guide for Vendors (Bright Blue)	1988; rev. 1990	XV
NCSC-TG-003	Discretionary Access Control in Trusted Systems	Sep. 1987	V
NCSC-TG-004	Glossary of Computer Security Terms	Oct. 1988	App. A
NCSC-TG-005	Trusted Network Interpretation (Red)	Jul. 1987	XI
NCSC-TG-006	Configuration Management in Trusted Systems	Mar. 1988	XIV
NCSC-TG-007	Design Documentation in Trusted Systems (Burgundy)	Oct. 1988	XV
NCSC-TG-008	Trusted Distribution in Trusted Systems (Lavender)	Dec. 1988	XIV
NCSC-TG-009	Computer Security Subsystem Interpretation (Venice Blue)	Sep. 1988	XII
NCSC-TG-010	Security Modeling in Trusted Systems	Oct. 1992	IV
NCSC-TG-011	Trusted Network Interpretation Environments Guideline (Red)	Aug. 1990	XI
NCSC-TG-013	Rating Maintenance Phase Program Document (Pink)	1989; v2 1995	XV
NCSC-TG-014	Guidelines for Formal Verification Systems (Purple)	Apr. 1989	XV
NCSC-TG-015	Trusted Facility Management (Brown)	1989	XVI
NCSC-TG-016	Guidelines for Writing Trusted Facility Manuals (Yellow-Green)	Oct. 1992	XVI
NCSC-TG-017	Identification and Authentication in Trusted Systems (Light Blue)	Sep. 1991	X
NCSC-TG-018	Object Reuse in Trusted Systems	Jul. 1992	VII
NCSC-TG-019	Trusted Product Evaluation Questionnaire (Blue)	v2 May 1992	XV
NCSC-TG-020-A	TRUSIX Rationale for Selecting ACL Features for UNIX	Aug. 1989	XII
NCSC-TG-021	Trusted Database Management System Interpretation	Apr. 1991	XII
NCSC-TG-022	Trusted Recovery in Trusted Systems (Yellow)	Dec. 1991	XIII
NCSC-TG-023	Security Testing and Test Documentation in Trusted Systems	Jul. 1993	XV
NCSC-TG-024	Procurement of Trusted Systems (multiple volumes)	1992–1994	XVII
NCSC-TG-025	Data Remanence in Automated Information Systems (Forest Green)	Sep. 1991	VIII
NCSC-TG-026	Writing the Security Features User's Guide (Hot Peach)	Sep. 1991	XVI
NCSC-TG-027	Information System Security Officer Responsibilities (Turquoise)	May 1992	XVI
NCSC-TG-028	Assessing Controlled Access Protection (Violet)	May 1992	XVI
NCSC-TG-029	Introduction to Certification and Accreditation (Blue)	Jan. 1994	XVII
NCSC-TG-030	Covert Channel Analysis of Trusted Systems (Light Pink)	Nov. 1993	IX

[29] ——, "CVE-2023-4969: GPU kernel implementations susceptible to memory leak," 2024, known as LeftoverLocals. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2023-4969

[30] ——, "CVE-2014-0160 (Heartbleed)," 2014. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2014-0160

[31] R. Chandramouli and E. Hibbard, "Guidelines for media sanitization," NIST, Special Publication 800-88 Rev. 2, Sep. 2025. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/88/r2/final

[32] "IEEE standard for sanitizing storage," IEEE Std 2883-2022, Aug. 2022.

[33] M. Wei, L. M. Grupp, F. E. Spada, and S. Swanson, "Reliably erasing data from flash-based solid state drives," in *Proc. 9th USENIX Conf. File Storage Technol. (FAST)*, 2011.

[34] J. A. Halderman *et al.*, "Lest we remember: Cold boot attacks on encryption keys," in *Proc. 17th USENIX Security Symp.*, 2008, pp. 45–60.

[35] R. A. Kemmerer, "Shared resource matrix methodology: An approach to identifying storage and timing channels," *ACM Trans. Comput. Syst.*, vol. 1, no. 3, pp. 256–277, Aug. 1983.

[36] P. C. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems," in *Advances in Cryptology—CRYPTO '96*, ser. Lecture Notes in Computer Science, vol. 1109, 1996, pp. 104–113.

[37] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, "Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds," in *Proc. 16th ACM Conf. Comput. Commun. Security (CCS)*, 2009, pp. 199–212.

[38] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack," in *Proc. 23rd USENIX Security Symp.*, 2014, pp. 719–732.

[39] Y. Kim *et al.*, "Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors," in *Proc. 41st Int. Symp. Comput. Archit. (ISCA)*, 2014, pp. 361–372.

[40] P. Kocher *et al.*, "Spectre attacks: Exploiting speculative execution," in *Proc. IEEE Symp. Security Privacy*, 2019, pp. 1–19.

[41] M. Lipp *et al.*, "Meltdown: Reading kernel memory from user space," in *Proc. 27th USENIX Security Symp.*, 2018, pp. 973–990.

[42] Y. Wang *et al.*, "Hertzbleed: Turning power side-channel attacks into remote timing attacks on x86," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 679–697.

[43] DoD Computer Security Center, "Department of defense password management guideline," Department of Defense Computer Security Center, Fort George G. Meade, MD, Tech. Rep. CSC-STD-002-85, Apr. 1985, "Green Book". [Online]. Available: https://irp.fas.org/nsa/rainbow/std002.htm

[44] National Computer Security Center, "A guide to understanding identification and authentication in trusted systems," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-017, Version 1, Sep. 1991, "Light Blue Book". [Online]. Available: https://irp.fas.org/nsa/rainbow/tg017.htm

[45] National Institute of Standards and Technology, "Digital identity guidelines: Authentication and authenticator management," NIST, Special Publication 800-63B-4, Jul. 2025.

[46] D. Balfanz *et al.*, "Web authentication: An API for accessing public key credentials – level 2," W3C Recommendation, Apr. 2021. [Online]. Available: https://www.w3.org/TR/webauthn-2/

[47] W3C Web Authentication Working Group, "Web authentication: An API for accessing public key credentials – level 3," W3C Recommendation, Aug. 2026. [Online]. Available: https://www.w3.org/TR/2026/REC-webauthn-3-20260825/

[48] FIDO Alliance, "Passkeys: Passwordless authentication," FIDO Alliance, online, 2026, accessed Nov. 5, 2026. [Online]. Available: https://fidoalliance.org/passkeys/

[49] C. Neuman, T. Yu, S. Hartman, and K. Raeburn, "The Kerberos network authentication service (V5)," RFC 4120, IETF, Jul. 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4120

[50] D. Hardt, "The OAuth 2.0 authorization framework," RFC 6749, IETF, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749

[51] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect core 1.0 incorporating errata set 2," OpenID Foundation, Dec. 2023. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html

[52] National Computer Security Center, "Trusted network interpretation," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-005, Version 1, Jul. 1987, "Red Book". [Online]. Available: https://irp.fas.org/nsa/rainbow/tg005.htm

[53] ——, "Trusted network interpretation environments guideline," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-011, Version 1, Aug. 1990, "Red Book". [Online]. Available: https://irp.fas.org/nsa/rainbow/tg011.htm

[54] National Institute of Standards and Technology, "Zero trust architecture," NIST, Special Publication 800-207, Aug. 2020.

[55] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," RFC 8446, IETF, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446

[56] S. Kent and K. Seo, "Security architecture for the Internet protocol," RFC 4301, IETF, Dec. 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4301

[57] National Computer Security Center, "Trusted database management system interpretation of the trusted computer system evaluation criteria," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-021, Version 1, Apr. 1991, "Purple" (Lavender) Book. [Online]. Available: https://irp.fas.org/nsa/rainbow/tg021.htm

[58] ——, "Inference and aggregation issues in secure database management systems," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC Technical Report 005, Volume 1/5, 1996, companion to NCSC-TG-021. [Online]. Available: https://wiretapped.mirrorservice.org/info/reference/ncsc-publications/NCSC-TR-005-1.txt

[59] ——, "Polyinstantiation issues in multilevel secure database management systems," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC Technical Report 005, Volume 3/5, May 1996, companion to NCSC-TG-021. [Online]. Available: https://wiretapped.mirrorservice.org/info/reference/ncsc-publications/NCSC-TR-005-3.txt

[60] ——, "Trusted UNIX working group (TRUSIX) rationale for selecting access control list features for the UNIX system," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-020-A, Version 1, Aug. 1989, "Silver" (Grey) Book. [Online]. Available: https://irp.fas.org/nsa/rainbow/tg020-a.htm

[61] ——, "Computer security subsystem interpretation of the trusted computer system evaluation criteria," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-009, Version 1, Sep. 1988, "Venice Blue Book". [Online]. Available: https://irp.fas.org/nsa/rainbow/tg009.htm

[62] PostgreSQL Global Development Group, "PostgreSQL 9.5 release notes," Jan. 2016, accessed Nov. 5, 2026. [Online]. Available: https://www.postgresql.org/docs/release/9.5.0/

[63] ——, "PostgreSQL 18 documentation, section 5.9: Row security policies," accessed Nov. 5, 2026. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[64] Microsoft, "Row-level security (SQL server)," Microsoft Learn, accessed Nov. 5, 2026. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security

[65] Oracle Corporation, *Oracle Label Security Administrator's Guide, 19c: Introduction to Oracle Label Security*, Oracle, accessed Nov. 5, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/19/olsag/introduction-to-oracle-label-security.html

[66] C. Dwork, "Differential privacy," in *Proc. 33rd Int. Colloq. Automata, Languages and Programming (ICALP)*, ser. Lecture Notes in Computer Science, vol. 4052. Springer, 2006, pp. 1–12.

[67] U.S. Census Bureau, "Understanding differential privacy," accessed Nov. 5, 2026. [Online]. Available: https://www.census.gov/programs-surveys/decennial-census/decade/2020/planning-management/process/disclosure-avoidance/differential-privacy.html

[68] OWASP Foundation, "SQL injection prevention cheat sheet," OWASP Cheat Sheet Series, accessed Nov. 5, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[69] A. Gruenbacher, "acl(5) – access control lists," Linux man-pages (acl project), accessed Nov. 5, 2026. [Online]. Available: https://man7.org/linux/man-pages/man5/acl.5.html

[70] D. Noveck and C. Lever, "Network file system (NFS) version 4 minor version 1 protocol," Internet Engineering Task Force, RFC 8881, Aug. 2020.

[71] The Linux Kernel Documentation, "Linux security module usage," accessed Nov. 5, 2026. [Online]. Available: https://docs.kernel.org/admin-guide/LSM/index.html

[72] National Information Assurance Partnership, *Protection Profile for General Purpose Operating Systems*, NIAP, Mar. 2026, version 5.0. [Online]. Available: https://commoncriteria.github.io/operatingsystem/main/operatingsystem.html

[73] National Computer Security Center, "A guide to understanding trusted recovery in trusted systems," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-022, Version 1, Dec. 1991, "Yellow Book"; primary author V. D. Gligor. [Online]. Available: https://irp.fas.org/nsa/rainbow/tg022.htm

[74] National Institute of Standards and Technology, "Platform firmware resiliency guidelines," NIST, Special Publication 800-193, May 2018.

[75] UEFI Forum, *Unified Extensible Firmware Interface (UEFI) Specification, Release 2.10, Ch. 32: Secure Boot and Driver Signing*, UEFI Forum, Inc., 2022. [Online]. Available: https://uefi.org/specs/UEFI/2.10/32_Secure_Boot_and_Driver_Signing.html

[76] Trusted Computing Group, *TCG PC Client Platform Firmware Profile Specification*, Trusted Computing Group, Dec. 2023, version 1.06, Revision 52. [Online]. Available: https://trustedcomputinggroup.org/resource/pc-client-specific-platform-firmware-profile-specification/

[77] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM Trans. Database Syst.*, vol. 17, no. 1, pp. 94–162, Mar. 1992.

[78] National Computer Security Center, "A guide to understanding configuration management in trusted systems," National Computer Security Center, Fort George G. Meade, MD, USA, Technical Guideline NCSC-TG-006-88, Version 1, Mar. 1988, library No. S-228,590. "Amber/Orange Book." [Online]. Available: https://irp.fas.org/nsa/rainbow/tg006.htm.

[79] ——, "A guide to understanding trusted distribution in trusted systems," National Computer Security Center, Fort George G. Meade, MD, USA, Technical Guideline NCSC-TG-008, Version 1, Dec. 1988, library No. S-228,592. "Lavender Book." [Online]. Available: https://irp.fas.org/nsa/rainbow/tg008.htm.

[80] K. Thompson, "Reflections on trusting trust," *Commun. ACM*, vol. 27, no. 8, pp. 761–763, Aug. 1984.

[81] FireEye/Mandiant, "Highly evasive attacker leverages SolarWinds supply chain to compromise multiple global victims with SUNBURST backdoor," Mandiant Threat Intelligence blog, Dec. 2020, accessed: Nov. 5, 2026. [Online]. Available: https://cloud.google.com/blog/topics/threat-intelligence/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor.

[82] CrowdStrike Intelligence Team, "SUNSPOT: An implant in the build process," CrowdStrike blog, Jan. 2021, accessed: Nov. 5, 2026. [Online]. Available: https://www.crowdstrike.com/en-us/blog/sunspot-malware-technical-analysis/.

[83] National Institute of Standards and Technology, "CVE-2024-3094 detail," National Vulnerability Database, Mar. 2024, accessed: Nov. 5, 2026. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2024-3094.

[84] A. Freund, "backdoor in upstream xz/liblzma leading to ssh server compromise," oss-security mailing list, Mar. 2024, accessed: Nov. 5, 2026. [Online]. Available: https://www.openwall.com/lists/oss-security/2024/03/29/4.

[85] Open Source Security Foundation, "xz backdoor CVE-2024-3094," OpenSSF blog, Mar. 2024, accessed: Nov. 5, 2026. [Online]. Available: https://openssf.org/blog/2024/03/30/xz-backdoor-cve-2024-3094/.

[86] SLSA Community, "SLSA specification v1.0: Security levels," OpenSSF, Apr. 2023, status now "Retired." Accessed: Nov. 5, 2026. [Online]. Available: https://slsa.dev/spec/v1.0/levels.

[87] ——, "SLSA specification v1.2," OpenSSF, Nov. 2025, includes Build and Source tracks. Accessed: Nov. 5, 2026. [Online]. Available: https://slsa.dev/spec/v1.2/.

[88] Reproducible Builds project, "Definitions," reproducible-builds.org, accessed: Nov. 5, 2026. [Online]. Available: https://reproducible-builds.org/docs/definition/.

[89] Sigstore project, "Sigstore overview," Sigstore documentation, accessed: Nov. 5, 2026. [Online]. Available: https://docs.sigstore.dev/about/overview/.

[90] J. J. R. Biden, "Executive order 14028: Improving the nation's cybersecurity," Federal Register, vol. 86, no. 93, pp. 26633–26647, May 2021, signed May 12, 2021; published May 17, 2021. [Online]. Available: https://www.federalregister.gov/d/2021-10460.

[91] National Telecommunications and Information Administration, "The minimum elements for a software bill of materials (SBOM)," U.S. Department of Commerce, Washington, DC, USA, Tech. Rep., Jul. 2021, [Online]. Available: https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf.

[92] National Institute of Standards and Technology, "Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities," NIST, Special Publication 800-218, Feb. 2022.

[93] National Computer Security Center, "A guide to understanding design documentation in trusted systems," National Computer Security Center, Fort George G. Meade, MD, USA, Technical Guideline NCSC-TG-007, Version 1, Oct. 1988, library No. S-228,591. "Burgundy Book." DTIC ADA392797. [Online]. Available: https://irp.fas.org/nsa/rainbow/tg007.htm.

[94] ——, "A guide to understanding security testing and test documentation in trusted systems," National Computer Security Center, Fort George G. Meade, MD, USA, Technical Guideline NCSC-TG-023, Version 1, Jul. 1993, library No. S-232,561; primary author V. D. Gligor. "Bright Orange Book." DTIC ADA277642. [Online]. Available: https://archive.org/details/DTIC_ADA277642.

[95] ——, "Guidelines for formal verification systems," National Computer Security Center, Fort George G. Meade, MD, USA, Technical Guideline NCSC-TG-014-89, Apr. 1989, library No. S-231,308. "Purple Book." [Online]. Available: https://irp.fas.org/nsa/rainbow/tg014.htm.

[96] ——, "NCSC endorsed tools list," Archival copy, Federation of American Scientists, entries for FDM 12.4 (evaluated Jan. 1988) and GVE 13.16 (evaluated Jul. 1987). [Online]. Available: https://irp.fas.org/nsa/ncsc_etl.htm.

[97] T. J. Bergendahl, R. J. Bottomly, R. J. Medlock, W. O. Sibert, and D. N. Stigdon, "Rating maintenance phase program document, version 2," National Computer Security Center, Fort George G. Meade, MD, USA, Technical Guideline NCSC-TG-013-95, Mar. 1995, library No. S-242,047. Supersedes Version 1 of 23 June 1989. "Pink Book." [Online]. Available: https://irp.fas.org/nsa/rainbow/tg013-2.htm.

[98] National Computer Security Center, "Trusted product evaluations: A guide for vendors," National Computer Security Center, Fort George G. Meade, MD, USA, Technical Guideline NCSC-TG-002, Version 1, Jun. 1990, library No. S-228,538. "Bright Blue Book." Earlier printing with foreword dated 1 Mar. 1988 at https://irp.fas.org/nsa/rainbow/tg002.htm. DTIC ADA392796. [Online]. Available: https://archive.org/details/DTIC_ADA392796.

[99] ——, "Trusted product evaluation questionnaire," National Computer Security Center, Fort George G. Meade, MD, USA, Technical Guideline NCSC-TG-019, Version 2, May 1992, "Blue Book." [Online]. Available: https://irp.fas.org/nsa/rainbow/tg019-2.htm.

[100] Common Criteria Development Board, "Assurance continuity: CCRA requirements," Common Criteria Recognition Arrangement, CCRA Supporting Document Version 3.1, Feb. 2024, [Online]. Available: https://www.commoncriteriaportal.org/files/operatingprocedures/CCDB-014-v3.1-2024-February-29-Final-Assurance_Continuity.pdf.

[101] Google, "OSS-Fuzz documentation," Google, accessed: Nov. 5, 2026. [Online]. Available: https://google.github.io/oss-fuzz/.

[102] The Rocq Development Team, "The Rocq prover 9.0," rocq-prover.org, Mar. 2025, formerly the Coq Proof Assistant. Accessed: Nov. 5, 2026. [Online]. Available: https://rocq-prover.org/releases/9.0.0.

[103] Lean FRO, "Lean programming language and theorem prover," lean-lang.org, accessed: Nov. 5, 2026. [Online]. Available: https://lean-lang.org/.

[104] National Computer Security Center, "A guide to understanding trusted facility management," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-015, Jun. 1989, version 1 ("Brown Book"); foreword dated 15 Aug. 1989. [Online]. Available: https://irp.fas.org/nsa/rainbow/tg015.htm.

[105] ——, "Guidelines for writing trusted facility manuals," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-016, Oct. 1992, version 1 ("Yellow-Green Book"). [Online]. Available: https://irp.fas.org/nsa/rainbow/tg016.htm.

[106] ——, "A guide to writing the security features user's guide for trusted systems," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-026, Sep. 1991, version 1 ("Hot Peach Book"). [Online]. Available: https://irp.fas.org/nsa/rainbow/tg026.htm.

[107] ——, "A guide to understanding information system security officer responsibilities for automated information systems," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-027, May 1992, version 1 ("Turquoise Book"). [Online]. Available: https://irp.fas.org/nsa/rainbow/tg027.htm.

[108] ——, "Assessing controlled access protection," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-028, May 1992, version 1 ("Violet Book"). [Online]. Available: https://irp.fas.org/nsa/rainbow/tg028.htm.

[109] Joint Task Force, "Security and privacy controls for information systems and organizations," National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication 800-53 Rev. 5, Sep. 2020, includes updates as of Dec. 10, 2020.

[110] Center for Internet Security, "CIS Benchmarks," Web page, accessed: Nov. 5, 2026. [Online]. Available: https://www.cisecurity.org/cis-benchmarks.

[111] U.S. Congress, "Federal agency responsibilities," 44 U.S.C. § 3554 (Federal Information Security Modernization Act of 2014, Pub. L. 113-283), Dec. 2014, [Online]. Available: https://www.law.cornell.edu/uscode/text/44/3554.

[112] DoD Computer Security Center, "Computer security requirements—guidance for applying the department of defense trusted computer system evaluation criteria in specific environments," DoD Computer Security Center, Fort George G. Meade, MD, Tech. Rep. CSC-STD-003-85, Jun. 1985, 25 June 1985 ("Light Yellow Book"). [Online]. Available: https://irp.fas.org/nsa/rainbow/std003.htm.

[113] ——, "Technical rationale behind CSC-STD-003-85: Computer security requirements—guidance for applying the department of defense trusted computer system evaluation criteria in specific environments," DoD Computer Security Center, Fort George G. Meade, MD, Tech. Rep. CSC-STD-004-85, Jun. 1985, 25 June 1985 ("Yellow Book"). [Online]. Available: https://irp.fas.org/nsa/rainbow/std004.htm.

[114] National Computer Security Center, "Introduction to certification and accreditation," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-029, Jan. 1994, version 1 ("Blue Book"). [Online]. Available: https://irp.fas.org/nsa/rainbow/tg029.htm.

[115] ——, "A guide to procurement of trusted systems: An introduction to procurement initiators on computer security requirements," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-024, Vol. 1/4, Dec. 1992, version 1 ("Purple Book"). [Online]. Available: https://www.csirt.org/color_%20books/NCSC-TG-024-1.pdf.

[116] ——, "A guide to procurement of trusted systems: Language for RFP specifications and statements of work—an aid to procurement initiators," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-024, Vol. 2/4, Jun. 1993, version 1, 30 June 1993. [Online]. Available: https://www.csirt.org/color_%20books/NCSC-TG-024-2.pdf.

[117] ——, "A guide to procurement of trusted systems: Computer security contract data requirements list and data item description tutorial," National Computer Security Center, Fort George G. Meade, MD, Tech. Rep. NCSC-TG-024, Vol. 3/4, Feb. 1994, version 1, 28 Feb. 1994. [Online]. Available: https://www.csirt.org/color_%20books/NCSC-TG-024-3.pdf.

[118] Joint Task Force, "Risk management framework for information systems and organizations: A system life cycle approach for security and privacy," National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication 800-37 Rev. 2, Dec. 2018.

[119] U.S. Department of Defense, "Risk management framework (RMF) for DoD information technology (IT)," DoD Instruction 8510.01, Mar. 2014, mar. 12, 2014. [Online]. Available: https://everyspec.com/DoD/DoD-PUBLICATIONS/DODI_8510-01_12MAR2014_29289/.

[120] National Institute of Standards and Technology, "Standards for security categorization of federal information and information systems," National Institute of Standards and Technology, Gaithersburg, MD, Federal Information Processing Standards Publication 199, Feb. 2004.

[121] S. VanRoekel, "Security authorization of information systems in cloud computing environments," Memorandum for Chief Information Officers, Office of Management and Budget, Executive Office of the President, Dec. 2011, dec. 8, 2011. [Online]. Available: https://obamawhitehouse.archives.gov/sites/default/files/omb/assets/egov_docs/fedrampmemo.pdf.

[122] Inspector General, U.S. Department of Defense, "Management advisory: The national information assurance partnership's evaluation and certification process for commercial off-the-shelf products," U.S. Department of Defense, Report DODIG-2025-165, Sep. 2025, redacted public version, Sept. 23, 2025. [Online]. Available: https://www.oversight.gov/sites/default/files/documents/reports/2025-09/DODIG-2025-165_Redacted_SECURE_0.pdf.

[123] J.-P. Aumasson, *Serious Cryptography*, 2nd ed. San Francisco, CA, USA: No Starch Press, 2024.

[124] D. Boneh and V. Shoup, "A graduate course in applied cryptography," Version 0.6, Jan. 2023. [Online]. Available: https://toc.cryptobook.us/

[125] M. Zalewski, *The Tangled Web*. San Francisco, CA, USA: No Starch Press, 2011.

[126] M. Dowd, J. McDonald, and J. Schuh, *The Art of Software Security Assessment*. Boston, MA, USA: Addison-Wesley, 2006.

[127] Microsoft Security Response Center, "A proactive approach to more secure code," Jul. 2019. [Online]. Available: https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code

[128] The Chromium Projects, "Memory safety." [Online]. Available: https://www.chromium.org/Home/chromium-security/memory-safety/

[129] OWASP Foundation, "OWASP top 10 for LLM applications 2025," 2025. [Online]. Available: https://genai.owasp.org/llm-top-10/

[130] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," NIST, Tech. Rep. NIST AI 100-1, Jan. 2023.

[131] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.

[132] National Institute of Standards and Technology, "Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile," NIST, Special Publication 800-61 Rev. 3, Apr. 2025.

[133] R. Bejtlich, *The Practice of Network Security Monitoring*. San Francisco, CA, USA: No Starch Press, 2013.

[134] A. Whitten and J. D. Tygar, "Why Johnny can't encrypt: A usability evaluation of PGP 5.0," in *Proc. 8th USENIX Security Symp.*, 1999, pp. 169–184.

[135] R. Anderson, "Why information security is hard—an economic perspective," in *Proc. 17th Annu. Comput. Security Appl. Conf. (ACSAC)*, 2001, pp. 358–365.

[136] B. W. Lampson, "Protection," *ACM SIGOPS Oper. Syst. Rev.*, vol. 8, no. 1, pp. 18–24, Jan. 1974.